



WEEK TWO

從系統雛形到產品骨架

用 Spec 管 Context，開始打造「可以被使用的產品」

在第二週，我們會一起完成 **App #2 | 內容管道儀表板**。

它不是為了做看板，而是為了讓你第一次真正理解：**當系統開始有「狀態」時，你必須學會管理 Context，否則 AI 一定會失控。**

這一週，你會第一次體驗到：同一筆資料，因為「狀態不同」，會出現在不同位置，而每一次互動，都會改變系統的整體行為。

別忘了新增本週實作資料夾：`vibe-coding-week2`

CHAPTER 0 回顧 WEEK 1	8
0.1 Week 1 我們到底做了什麼？（用產品語言重新看一次）	9
第一件事：你幫系統畫出了「邊界」	9
第二件事：你幫系統定義了「行為路徑」	9
第三件事：你第一次握住了「控制權」	9
0.2 Spec-driven 到底在「驅動」什麼？	10
0.3 為什麼 Week 2 一定會更容易「爆掉」？	11
0.4 本週你要扮演的角色，跟上週不一樣了	12
0.5 Week 2 的工作方式（很重要，請先看）	12
0.6 從這一章開始，你會怎麼用 Amazon Q？	13
0.7 帶著這個心態，進入 Chapter 1	14
CHAPTER 1 產品為什麼一定會有「狀態」	15
1.1 為什麼沒有狀態的系統，只能是 Demo	15
1.2 什麼是「狀態」？先不要用工程腦理解	16
1.3 為什麼「狀態一出現」，Context 就會變得很危險	17
1.4 這就是為什麼「App #2」一定從狀態開始	19
1.5 本章你「不需要」先會的事情	19
1.6 本章的 Amazon Q 使用方式（先確認，不動手）	20
1.7 帶著這些理解，進入 Chapter 2	21
CHAPTER 2 CONTEXT-DRIVEN 的真相	22
2.1 先破除一個迷思：AI 沒有「記憶斷點」	22
2.2 為什麼我們要先準備一個 Demo 專案？	23
2.3 為什麼產品型專案，Context 特別容易失控？	24
2.4 Token 爆炸，其實是 Context 不穩定的症狀	26

2.5 Context-driven 本身不是問題，失控才是	27
2.6 Spec-driven 的真正角色：不是命令，是護欄	28
2.7 本章你應該開始具備的判斷力	29
2.8 Chapter 2 的 Amazon Q 使用方式（分析用，不實作）	30
2.9 帶著這個理解，進入 Chapter 3	31
CHAPTER 3 SPEC-DRIVEN 的真正用途	32
3.1 先校正一個錯誤期待：Spec 不是一次寫完的	32
3.2 Spec 要控制的不是功能，而是「理解範圍」	33
3.3 產品級 Spec 的四個必要區塊（Week 2 版本）	34
3.4 現在，把 Spec 寫回 demo 專案裡	36
3.5 Amazon Q 使用方式（對齊理解，不直接實作）	37
3.5-EX Spec 對齊實測：讓 Amazon Q 幫你照鏡子	38
3.6 為什麼「Spec 對齊」一定要在寫程式前？	42
3.7 本章你應該達成的狀態	43
3.8 帶著這份 Spec，進入 Chapter 4	43
CHAPTER 4 產品級三原則	44
4.1 為什麼產品一進實作，就特別容易炸？	44
4.2 原則一：Backend First（依賴順序控制）	45
4.3 原則二：Iterative（小步快跑，控制變動幅度）	47
4.4 原則三：Atomic Edit（控制傷害範圍）	49
4.5 三原則其實是在做同一件事	50
4.6 如果你違反其中一個，會發生什麼事？	50
4.7 本章結束時，你應該站在這裡	50

4.8 帶著三原則，進入 Chapter 5	51
CHAPTER 5 APP #2 的產品定位	52
5.1 為什麼「能用」不等於「會被用」？	52
5.2 App #2 的真正身份，不是一個看板	53
5.3 產品定位的第一步：先決定「非誰不可」	53
5.4 從功能思維，轉成「場景思維」	54
5.5 App #2 的核心使用場景（Week 2 版）	55
5.6 這時候，你才有資格問：要不要加功能？	56
5.7 Chapter 5 的 Amazon Q 使用方式（產品共識驗證）	57
5.8 你現在應該開始有的感覺	57
5.9 為什麼我們在 Week 2 停在這裡？	58
5.10 帶著產品定位，進入 Chapter 6	58
CHAPTER 6 APP #2 教學用 SPEC	59
6.1 Spec 使用說明（請先讀）	59
6.2 Spec 檔案位置（全班統一）	59
6.3 App #2 Spec（學員填空版）	60
6.4 這份 Spec 在教你什麼？	63
6.5 Chapter 6 專用 Amazon Q 指令（Spec 驗收）	64
6.6 為什麼我們用「學員填空版」？	64
6.7 Chapter 6 結束時，你應該站在這裡	65
6.8 下一步預告（Week 3）	65
CHAPTER 7A 本地展示與交付	66

7A.1 為什麼 Week 2 不急著部署？	66
7A.2 Week 2 的「完成定義」是什麼？	67
7A.3 本地展示方式 (Live Server)	67
7A.4 Demo Script (學員展示用)	68
7A.5 交付內容清單 (Week 2 交件包)	69
7A.6 為什麼「交付資料夾」很重要？	70
7A.7 Week 2 結束時，你應該站在這裡	70
CHAPTER 7B (前導) 上線與商業化的準備	71
7B.1 先講結論：部署不是只有一種路	71
7B.2 為什麼我們先選 Firebase Hosting？	72
7B.3 Cloud Run 在產品路線中的角色	72
7B.4 從部署走向商業化 (Week 3 將做的事)	72
7B.5 給進階學員的一句話	73
CHAPTER 8 狀態升級與互動進階	74
8.1 為什麼互動升級一定要「延後」？	74
8.2 互動升級的第一個關卡：狀態「回退」要不要存在？	75
8.3 狀態升級的核心原則 (仍然適用三原則)	75
8.4 拖拉 (Drag & Drop) 為什麼是「最後才加」？	77
8.5 示範架構：互動 ≠ 狀態邏輯	78
8.6 Chapter 8 的 Amazon Q 使用方式 (示範用)	79
8.7 這一章，你要「帶走什麼」？	81
8.8 Chapter 8 的定位總結	81
8.9 銜接下一章	81

CHAPTER 9 資料持久化與多裝置	82
9.1 為什麼資料持久化不是「順手加一下」？	82
9.2 資料持久化，會讓系統多背一個責任	83
9.3 三種常見的資料持久化層級（產品視角）	84
9.4 在 App #2，我們為什麼「只示範 localStorage」？	85
9.5 資料持久化，對 Spec 的影響	85
9.6 Amazon Q 使用方式（技術選型顧問）	86
9.7 示範段：localStorage 接入的正確順序（講師示範）	86
9.8 常見錯誤示範	87
9.9 本章結束時，你應該理解的三件事	87
9.10 銜接下一章	87
Chapter 10 真實上線後的產品調整	88
10.1 上線後第一個會壞的，通常不是技術	88
10.2 「回饋」不是意見，是行為的痕跡	89
10.3 為什麼這些問題「不能直接改程式」？	89
10.4 上線後的正確調整順序（非常重要）	90
10.5 Spec 在這一章的角色改變了	90
10.6 Amazon Q 使用方式（回饋分析員）	91
10.7 常見的「錯誤修法」	91
10.8 一個健康的產品調整循環	92
10.9 為什麼這一章，是 AI 課程的分水嶺？	92
10.10 銜接最後一章	93
CHAPTER 11 AI × 產品的長期協作模式	94

11.1 AI 在產品生命週期中的三種角色	94
11.2 真正成熟的協作，不是 Prompt，而是「位置」	95
11.3 把 AI 從「救火隊」拉回「標準流程」	95
11.4 Spec × AI：一個會一起老的組合	96
11.5 長期協作的最小閉環（請記住這個循環）	97
11.6 為什麼這一章，才是真正的「AI 素養」	98
11.7 一段現實提醒（很重要）	98
11.8 你現在站在哪裡？	99
11.10 真正的結尾（不是下課）	99
WEEK 2 總結：我們這一週到底做了什麼？	100
一、Week 2 的真正主題，不是功能	100
二、你在 Week 2 學到的三個關鍵能力	101
三、為什麼我們刻意停在「本地 Demo」？	102
四、如果你現在有這些感覺，是對的	102
Week 3 預告	103
附錄：一頁 WEEK 2 → WEEK 3 能力轉換圖	105
附錄：WEEK 2 課後反思題	107
附錄：WEEK 2 課後 AMAZON Q 指令卡	108

Chapter 0 回顧 Week 1

你其實已經在「控制 AI」，只是你還沒發現
把 Spec 從文件，升級成「Context 控制器」

在進入 Week 2 之前，我們要先做一件非常重要、但常被跳過的事：
確認你在 Week 1 學到的，不只是寫文件，而是一種「讓 AI 聽話」的能力。

如果你沒有先把這件事對齊，那 Week 2 一開始打造產品時，你會很容易出現這種感覺：

- 我好像都懂，但 AI 一改就歪
- 怎麼功能一多，整個系統就亂了
- 是不是我指令下得不夠好？

其實都不是。

你只是還沒意識到 Week 1 的能力，現在要正式升級了。

0.1 Week 1 我們到底做了什麼？（用產品語言重新看一次）

在 Week 1，你表面上做的是：

- 寫 Spec
- 定 Schema
- 畫 User Flow
- 用 AI 做出一個可以跑的小系統

但如果我們用「產品」的角度重新看，你其實完成了三件非常關鍵的事：

第一件事：你幫系統畫出了「邊界」

透過 Schema，你不是在定資料表，而是在告訴 AI：

- ☐ 哪些資料存在
- ☐ 哪些資料現在不准出現

這一步的本質是：限制 AI 的想像空間。

第二件事：你幫系統定義了「行為路徑」

透過 User Flow，你不是在交流程作業，而是在建立一張「出事時可以回頭看的地圖」。

這代表一件事：當系統怪怪的，你不是重寫，而是知道「該回到哪一步檢查」

第三件事：你第一次握住了「控制權」

Spec 文件的真正功能，不是記錄需求，而是讓 AI 在一個被限制、被定義過的世界裡工作。

到這裡為止，你已經不是「跟 AI 聊天」，而是開始在管理 AI 的工作方式。

0.2 Spec-driven 到底在「驅動」什麼？

在 Week 1，我們一直說這門課是 Spec-driven，但到了 Week 2，你會開始看到另一個詞：Context-driven。

很多人會在這裡開始混亂，所以我們直接一次講清楚。

先說結論（請先記住這一句）

AI 永遠是 Context-driven。

Spec-driven 不是取代它，而是用來「設計 Context」。

那什麼是 Context？

對 AI IDE（包含 Amazon Q）來說，Context 包含：

- ☐ 目前專案的資料夾結構
- ☐ 每一個檔案的內容
- ☐ 你剛剛做了哪些修改
- ☐ 你現在下了什麼指令
- ☐ 哪些地方是「最近被動過的」

AI 不是只看你這一句話，而是看「整個專案目前長什麼樣子」。

那 Spec 在幹嘛？Spec 的工作只有一個核心任務：

讓 AI 每次看到的 Context，都長得差不多。

你在 Week 1 其實已經做了三件事：

- 壓縮 Context（讓選項變少）
- 穩定 Context（讓世界觀固定）
- 鎖定 Context（讓 AI 不亂動）

只是你當時還沒把這件事「講清楚」。

0.3 為什麼 Week 2 一定會更容易「爆掉」？

因為從這一週開始，系統會多一個新元素：狀態（State）

Week 1 的資料是「靜態的」

- 新增資料
- 顯示資料
- 刪掉再來

Context 變化很小，AI 比較不容易失控。

Week 2 的資料會「移動」

- 同一筆資料
- 會因為狀態不同
- 出現在不同位置
- 被不同邏輯處理

這代表什麼？Context 會頻繁變動，而且是連續變動。

如果你沒有用 Spec 把規則寫清楚，AI 就會開始「合理補完」，然後幫你把系統補壞。

0.4 本週你要扮演的角色，跟上週不一樣了

請你先暫時放下「我是新手」這個身份。

從 Week 2 開始，你的角色會變成：正在打造一個產品雛形的人。

這代表三件事開始變得重要：

1. 你做的每個決定，都會影響後面能不能上線
2. 你加的每個功能，都可能影響系統穩定性
3. 你必須開始「選擇不做什麼」

別把這當作壓力，這是你開始真的在做產品的證據。

0.5 Week 2 的工作方式（很重要，請先看）

在本週所有實作中，我們會統一使用：`vibe-coding-week2`

這不是格式規定，而是 **Context** 管理的一部分。

- Week 1 的專案，不要混進來
- Week 2 的所有嘗試，都在同一個世界裡完成
- 出問題時，你知道你是在「哪一週的 Context」

0.6 從這一章開始，你會怎麼用 Amazon Q？

從 Week 2 開始，你會明顯感覺到：不能再用「幫我做一個 XXX」這種指令了。但你也不用擔心，因為我們會一起把指令變得很清楚。

本章對應的 Amazon Q 使用原則（先記住）

在 Chapter 0 ~ Chapter 2，你只會用 Amazon Q 做三件事：

✓ 理解現況

幫我總結目前專案的結構與資料流

✓ 對齊理解

請你依照我提供的 Spec，確認你對系統的理解是否正確

✗ 不做任何實作修改

這一段不讓 AI 動手，只讓它「確認 Context」。

範例 Amazon Q 指令（Chapter 0 專用）

你現在可以照貼、照用：

先不要修改任何程式碼。

閱讀目前 vibe-coding-week2 專案的資料夾與檔案結構，用條列方式說明你理解的系統目標、主要資料結構與目前流程。

如果有不確定或尚未定義的地方，請明確標出。

這一類指令的目的只有一個：**確認 AI 跟你活在同一個 Context 裡。**

0.7 帶著這個心態，進入 Chapter 1

在進入下一章之前，請你記住這三句話：

- 你不是來「多寫功能」的
- 你是來「讓系統開始穩定運作」的
- Spec 不是作業，是你的煞車系統

接下來，我們會從一個產品一定會遇到的問題開始：為什麼只要系統想被使用，就一定會出現「狀態」？

Chapter 1 產品為什麼一定會有「狀態」

從「能跑的系統」，走向「會被使用的產品」

在 Week 1，你已經成功做出一個可以運作的系統，它可以新增資料、顯示資料，而且流程是通的。但從 Week 2 開始，我們要面對一個新的現實：**只要一個系統想被「持續使用」，它就一定會出現「狀態」。**

這一章不是在教技術名詞，而是在幫你建立一個非常重要的產品直覺。

1.1 為什麼沒有狀態的系統，只能是 Demo

先回想一下你在 Week 1 做的事情：輸入資料、存起來，最後顯示出來。

這樣的系統，有沒有用？有。但它有一個很明顯的限制：**它不知道事情「現在進行到哪裡」。**

Demo 系統 vs 產品系統的差別

如果我們用一句話區分：

- Demo 系統關心的是
「資料有沒有成功存進去」
- 產品系統關心的是
「這件事現在卡在哪一個階段」

一旦有人開始真的使用你的系統，就一定會出現這些問題：

- 這個東西是還沒開始？
- 正在處理中？
- 已經完成了？
- 還需要我再做什麼嗎？

這些問題，**不是 UI 問題**，而是「狀態」問題。

1.2 什麼是「狀態」？先不要用工程腦理解

在這門課裡，我們先不要把狀態想成技術名詞。

你可以先用一個很生活化的方式理解它：**狀態 = 一件事情「現在的立場」**。

舉例來說：

- 一篇內容
 - 還沒開始寫
 - 正在編輯中
 - 已經完成
- 一個申請
 - 已送出
 - 審核中
 - 已通過
- 一個任務
 - 待辦
 - 進行中
 - 已完成

你會發現一件事：**狀態不是資料的屬性，而是「行為的階段」**。

1.3 為什麼「狀態一出現」，Context 就會變得很危險

這一節很重要，因為它會直接影響你後面怎麼用 AI。

在沒有狀態的系統裡：

- 資料新增一次
- 顯示一次
- Context 變化很小

但只要你引入狀態，事情就不一樣了。

狀態會讓系統發生三件事：

第一件事：資料會開始「移動」

同一筆資料，可能會：

- 原本顯示在 A 區
- 狀態改變後，出現在 B 區
- 在 A 區消失

這代表什麼？畫面不再固定對應某一筆資料。

第二件事：畫面開始依賴邏輯，而不是資料本身

你看到的不是「全部資料」，而是「符合某個狀態的資料」。

這代表：UI 是結果，不是來源。

第三件事：Context 會頻繁、連續地變動

每一次狀態切換，都會：

- 改資料
- 改畫面
- 改流程

如果這些規則沒有先寫清楚，AI 就會開始補邏輯、補流程、補假設。

然後你會看到熟悉的畫面：

- token 開始暴增
- 改一個地方，其他地方壞掉
- AI 看起來很努力，但系統越來越亂

1.4 這就是為什麼「App #2」一定從狀態開始

在 Week 2，我們選擇的練習專案是：App #2 內容管道儀表板 (Content Pipeline Board)，它表面上看起來像一個看板，但真正的學習重點只有一個：
如何在「狀態不斷變動」的情況下，讓系統還是穩定的。

這個 App 會逼你面對三件事：

1. 一筆資料只能有一個真實狀態
2. 狀態改變，畫面就必須跟著改
3. 所有狀態轉移，都必須是「被允許的」

這正是產品世界每天都在發生的事。

1.5 本章你「不需要」先會的事情

請先放心，這一章刻意不做以下事情：

- 不談複雜的前端狀態管理
- 不談框架或套件
- 不談效能或最佳化
- 不談拖拉或動畫

因為在 Week 2，我們只關心一件事：

狀態是否清楚、資料是否一致、Context 是否可控

1.6 本章的 Amazon Q 使用方式（先確認，不動手）

在 Chapter 1，我們還不會請 AI 幫你實作功能。

我們只用 Amazon Q 做一件事：確認「我們對狀態的理解是否一致」。

Chapter 1 專用 Amazon Q 指令（理解用）

在 vibe-coding-week2 專案中，直接使用以下指令：

先不要修改任何程式碼。

我正在設計一個會有狀態的產品系統，
狀態包含：To Do、In Progress、Done。

用「產品行為」而不是技術實作的角度，
說明在這種系統中：

1. 狀態存在的意義是什麼
2. 狀態改變時，系統行為會有哪些改變
3. 如果狀態規則沒有定義清楚，最容易出現哪些問題

只做分析與說明，不要提供任何程式碼。

這個指令的目的不是得到標準答案，而是讓你確認：

AI 跟你是否站在同一個產品理解上

1.7 帶著這些理解，進入 Chapter 2

如果你現在能回答這三個問題：

- 為什麼產品一定會有狀態？
- 為什麼狀態會讓系統更容易亂？
- 為什麼這一週一定要用 Spec 管 Context？

那代表你已經準備好進入下一章。

在 Chapter 2，我們要正面拆解一個問題：

當系統開始像產品一樣成長時，AI 為什麼會特別容易失控？

Chapter 2 Context-driven 的真相

為什麼系統一開始像天使，後來卻越改越失控？

在 Chapter 1，我們知道了一件事：只要系統開始有狀態，Context 就會頻繁變動。

那接下來這個問題就一定會出現：為什麼一樣用 AI，小練習都很順，一變成產品型 App 就開始亂？這一章，我們要把這件事一次說清楚。

2.1 先破除一個迷思：AI 沒有「記憶斷點」

很多人以為 AI 是這樣工作的：

- 我現在下指令
- AI 只針對這個指令處理
- 做完就結束

但事實完全不是。

AI IDE 的真實工作方式

在 Amazon Q、Cursor、Claude Code、Gemini Code Assist 這類工具中，AI 是這樣理解你的世界的：

我現在看到的所有東西，都是我做判斷的依據。

這些「所有東西」包含：

- 專案目前的資料夾結構
- 所有檔案的內容
- 最近修改過哪些地方
- 你現在這一句指令
- 你前幾步到底做了什麼

沒有重來鍵，也沒有自動清空 Context

2.2 為什麼我們要先準備一個 Demo 專案？

在進入產品開發之前，我們刻意先建立了一個不完整、但方向明確的 demo 專案 (`vibe-coding-week2`)。

你現在看到的這個專案裡：

- 有資料夾結構
- 有初步 Spec
- 有狀態欄位
- 但沒有完整邏輯

這不是偷懶，而是刻意設計。

原因只有一個：

**Context 不是等你「寫完系統」才出現
而是從你建立第一個檔案就開始存在**

這個 demo 專案的用途，是讓你跟 AI 站在同一個「世界畫面」裡。

請下載 `demo.zip` 至 `vibe-coding-week2` 資料夾，解壓縮。

2.3 為什麼產品型專案，Context 特別容易失控？

因為產品型專案，跟小練習有三個根本差異。

差異一：檔案數量會快速增加

一開始你可能只有：

- 一個 HTML
- 一個 JS
- 一個資料結構

但一進入產品思維，很快就會出現：

- 多個邏輯檔
- 多個狀態處理
- 多個顯示區塊

Context 不是線性增加，而是指數成長。

差異二：你會頻繁「回頭改」

產品不是一次寫完的。

你一定會：

- 改流程
- 改狀態
- 改命名
- 改結構

這代表：AI 看到的是「被動過很多次的世界」

如果你沒有告訴它哪些是穩定的、哪些是可變的，它就只能全部重新推理。

差異三：狀態會互相影響

在產品裡：

- 改一個狀態
- 往往會影響三個畫面
- 牽動兩條流程

如果這些關係沒有被寫清楚，AI 很容易做出「局部正確、整體錯誤」的修改。

2.4 Token 爆炸，其實是 Context 不穩定的症狀

這裡請你先記住一個很重要的判斷方式：

Token 爆，不是因為你話多，而是因為 AI 必須「重新理解整個世界」

常見的失控情境

你可能做過這些事：

- ☐ 幫我調整一下狀態顯示
- ☐ 我想稍微改一下流程
- ☐ 這裡怪怪的，幫我修一下

對你來說是小改，對 AI 來說卻是：

我是不是要重新推理所有可能影響到的地方？

於是它開始：

- 回顧整個專案
- 嘗試補齊缺失邏輯
- 預防你沒講清楚的情境

結果就是：

- Token 飆高
- 修改範圍失控
- 改 A 壞 B

2.5 Context-driven 本身不是問題，失控才是

這一點非常重要，請不要誤會。

Context-driven 不是缺點，而是 AI 能工作的前提

問題從來不是：

- AI 太聰明
- AI 愛亂改
- AI 不聽話

真正的問題是：Context 沒有被人類設計過。

沒有 Spec 的 Context，會發生什麼事？

- ☐ AI 只能猜你的產品意圖
- ☐ 每一次修改，都是一次「世界觀重建」
- ☐ 推理成本越來越高
- ☐ 穩定性越來越低

這時候你會有一個錯覺：「是不是這個工具不適合我？」

但事實是：任何 Context-driven 工具，在沒有 Spec 時，結果都一樣。

2.6 Spec-driven 的真正角色：不是命令，是護欄

現在我們把 Chapter 1 跟 Chapter 2 接起來。請記住這一句話：

Spec 不是拿來叫 AI 做事的

Spec 是用來限制 AI「怎麼想事情」的

Spec 在 Context 中扮演的三個角色

第一個角色：縮小思考空間

- 明確定義資料結構
- 明確定義狀態集合
- AI 不用再猜「還有沒有其他可能」

第二個角色：固定世界觀

- 什麼是核心流程
- 什麼不是這一版要處理的
- 每次修改都在同一個框架裡進行

第三個角色：標示「禁止區域」

- 哪些檔案不能動
- 哪些邏輯暫時不碰
- 哪些狀態不允許轉移

這會直接降低 AI 的「過度補完」。

2.7 本章你應該開始具備的判斷力

在進入下一章之前，你應該開始能判斷這些狀況：

- 現在 AI 是在「理解清楚後修改」
- 還是在「邊猜邊補」？
- 這次修改，是不是動到太多 Context？
- 我是不是該先停下來補 Spec？

如果你開始有這些念頭，代表你已經從「使用者」進化成「管理者」。

2.8 Chapter 2 的 Amazon Q 使用方式 (分析用，不實作)

在 Chapter 2，我們仍然不讓 AI 動任何程式碼。我們只用 Amazon Q 來做一件事：

幫你看清楚目前的 Context 長什麼樣子

Chapter 2 專用 Amazon Q 指令 (Context 盤點)

請在 `vibe-coding-week2` 專案中，直接使用以下指令：

先不要修改任何程式碼。

請你以「Context-driven AI」的角度，盤點目前 `vibe-coding-week2` 專案中，有哪些 Context 來源 (例如檔案結構、資料結構、狀態邏輯、流程假設)。

請說明：

1. 哪些 Context 是明確定義過的
2. 哪些 Context 可能需要由你自行推測
3. 如果我要開始加入「狀態管理」，哪些地方最容易出現理解不一致

只做分析，不要提供任何實作建議或程式碼。

這個指令的目的只有一個：讓你知道 AI 現在是怎麼「看你的專案」的。

2.9 帶著這個理解，進入 Chapter 3

到目前為止，我們已經確認三件事：

1. 產品一定會有狀態
2. 狀態會讓 Context 快速變動
3. Context-driven 本身沒錯，沒被管理才會失控

你不再是盲寫系統，而是開始管理「理解落差」。

接下來的問題只剩一個：

那 Spec，到底要怎麼寫，才能真的「管得住 Context」？

這正是 Chapter 3 要做的事。

Chapter 3 Spec-driven 的真正用途

Spec 不是文件，而是你用來「接管 Context」的控制器

在 Chapter 2，你已經做了一件很重要的事：

你第一次看清楚了 AI 現在是怎麼理解你的專案

你也應該已經發現一件事：

- AI 並沒有亂來
- 它只是「補齊了你沒說清楚的地方」

所以，問題現在變得非常具體：那我要怎麼寫 Spec，才能讓 AI 不用猜？

這一章，我們要讓 Spec 從「說明文件」，正式升級成 Context 的控制器。

3.1 先校正一個錯誤期待：Spec 不是一次寫完的

很多人聽到 Spec-driven，腦中會出現這個畫面：

- ☐ 先寫一份超完整 Spec
- ☐ 再交給 AI 一次做完
- ☐ 中途不再動

這個想像，在產品世界裡幾乎不可能成立。

產品型 Spec 的真實樣子

在真實產品開發中，Spec 通常是：

- 一開始很小
- 隨著系統成長而補齊
- 每次只解決「現在這一段 Context」

所以在 Week 2，我們用的是這個心法：Spec 不是藍圖，而是「護欄」。

3.2 Spec 要控制的不是功能，而是「理解範圍」

在 Chapter 2，你已經讓 Amazon Q 幫你盤點過：

- 哪些地方是明確的
- 哪些地方是它必須自行推測的

Spec 的第一個任務，就是把「必須推測的地方」一個一個消掉。

用 App #2 的例子來看

目前在 demo 專案中，你已經有：

- ☐ 狀態欄位 (status)
- ☐ 三個狀態名稱
- ☐ 初步資料結構

但還有很多地方沒寫清楚，例如：

- ☐ 狀態可以怎麼轉？
- ☐ 哪些轉移是不允許的？
- ☐ 狀態改變時，畫面應該發生什麼事？

這些「沒寫的地方」，就是 Context 最容易失控的地方。

3.3 產品級 Spec 的四個必要區塊 (Week 2 版本)

從這一章開始，我們的 Spec 會有一個明確結構，不是為了漂亮，而是為了讓 AI 的理解穩定。

區塊一：系統目標 (System Goal)

請你用一句話回答：這個系統「現在」要解決什麼問題？

⚠ 請注意「現在」兩個字。

Week 2 的目標不是未來藍圖，而是：

- 管理內容狀態
- 確保資料一致
- 支援後續擴充

區塊二：資料與狀態定義 (Data & State)

這一段不是資料庫設計，而是語意設計。

你要回答的是：

- 一筆資料，最少包含什麼？
- status 有哪些合法值？
- 是否允許自由新增狀態？

如果這裡沒寫清楚，AI 就一定會「好心幫你補」。

區塊三：允許的流程 (Allowed Flow)

這一段非常關鍵，因為它直接影響 AI 的修改行為。

你要明確寫出：

- 哪些狀態可以轉到哪些狀態
- 哪些轉移是禁止的
- 一次操作只做一件事

這會大幅降低「牽一髮動全身」的修改。

區塊四：不在本週範圍內 (Out of Scope)

這不是偷懶，而是保命。

請你明確寫出：

- 這一週不做什麼
- 哪些功能暫時不碰
- 哪些結構不能動

Spec 不寫「不做什麼」，Context 一定會膨脹。

3.4 現在，把 Spec 寫回 demo 專案裡

接下來這一步很重要：Spec 不是寫在腦中，也不是寫在投影片裡。

它必須**存在於專案本身**，成為 Context 的一部分。

Week 2 Spec 放置位置 (統一)

請在以下位置撰寫：

```
vibe-coding-week2/  
└─ spec/  
    └─ app-spec.md
```

這代表一件事：AI 以後每次看專案，都會看到這份 Spec。

3.5 Amazon Q 使用方式 (對齊理解，不直接實作)

從這一章開始，Amazon Q 可以「開始參與」，但角色仍然是：理解對齊者，不是實作者。

Chapter 3 專用 Amazon Q 指令 (Spec 對齊)

請在你完成 spec/app-spec.md 初稿後，使用以下指令：

請先不要修改任何程式碼。

我在 vibe-coding-week2/spec/app-spec.md 中，新增了一份 App #2 的 Spec。

請你閱讀這份 Spec，並用條列方式說明：

1. 你理解的系統目標是什麼
2. 你理解的資料結構與狀態規則
3. 你認為哪些行為是被允許的
4. 你認為哪些行為應該被禁止

如果你發現任何規則不明確、可能產生多種解讀的地方，請直接指出來，但不要自行補規則。

這一步的目的只有一個：確認 AI 的理解，是否真的被 Spec 鎖住了。

3.5-EX Spec 對齊實測：讓 Amazon Q 幫你照鏡子

在這一節，我們不只是「下指令」，而是要讓你親眼看到：

- Spec 寫得清楚時，AI 會怎麼回應
- Spec 不夠清楚時，AI 會卡在哪裡

Step 1 | 先準備一份「刻意不完美，但可用的 Spec」

請先在專案中建立或補齊以下檔案：`vibe-coding-week2/spec/app-spec.md`，並填入以下範例內容（你可以直接貼上，先不要改）：

```
# App #2 – Content Pipeline Board (Week 2 Spec)

## 1. System Goal

This app manages content items through a simple pipeline.
Each item has a single status that represents its current stage.

The goal for Week 2 is:
- Keep state changes predictable
- Ensure data consistency
- Avoid complex features

## 2. Data & State

Each content item includes:
- id (number)
- title (string)
- status (string)

Allowed status values:
- todo
```

- in-progress
- done

No other status values are allowed in Week 2.

3. Allowed Flow

- An item can move from `todo` to `in-progress`
- An item can move from `in-progress` to `done`
- Direct move from `todo` to `done` is not allowed
- Items in `done` cannot change status

Each action updates only one item at a time.

4. Out of Scope

- Drag and drop
- User authentication
- Editing item content
- Adding new status values

這份 Spec 不是最終版，它的角色是「讓你練習對齊 Context」。

Step 2 | 執行 Chapter 3 的 Amazon Q 對齊指令

現在，請你在 VS Code 中對 Amazon Q 輸入以下指令：

請先不要修改任何程式碼。

我在 `vibe-coding-week2/spec/app-spec.md` 中
新增了一份 App #2 的 Spec。

請你閱讀這份 Spec，並用條列方式說明：

1. 你理解的系統目標是什麼
2. 你理解的資料結構與狀態規則
3. 你認為哪些行為是被允許的
4. 你認為哪些行為應該被禁止

如果你發現任何規則不明確、可能產生多種解讀的地方，請直接指出來，但不要自行補規則。

Step 3 | 你應該看到什麼樣的回覆，才算「對齊成功」？

當 Spec 寫得「夠用」時，Amazon Q 的回覆通常會有以下特徵：

對齊成功的訊號

- 它用跟你 Spec 幾乎一樣的語言描述系統
- 它不會新增新的狀態名稱
- 它會清楚說出哪些轉移被禁止
- 它會指出 Out of Scope，而不是幫你補功能

例如，你可能會看到這樣的描述（示意）：

The system manages content items through a linear pipeline with three allowed statuses: todo, in-progress, and done.

Items can only move forward one step at a time, and once an item reaches done, no further changes are allowed.

這代表一件事：AI 現在看到的世界，跟你腦中的是同一個世界。

Step 4 | 哪些回覆，是「Spec 還沒鎖住 Context」的警訊？

如果你看到 Q 出現以下行為，請先不要急著修程式：

危險訊號

- 它開始問：Can items move back to previous states?
- 它提到：It might be useful to allow editing...
- 它假設：Users may want to add custom statuses...

這些不是 Q 亂來，而是：你的 Spec 還留了太多「可以合理推測」的空間。

這時你該做的不是改程式，而是：

- ☐ 回到 app-spec.md
- ☐ 補一句「不允許」
- ☐ 再跑一次對齊指令

Step 5 | 這個實測在教你什麼？

請你停下來想一件事：剛剛這個過程，AI 其實完全沒有寫任何程式碼。

但你已經做到三件事：

1. 降低 Context 的不確定性
2. 防止 AI 提前補功能
3. 為後續實作建立安全邊界

這正是 Spec-driven 的真正價值。

3.6 為什麼「Spec 對齊」一定要在寫程式前？

因為一旦你開始實作：

- 檔案數會變多
- Context 會變複雜
- 修改成本會變高

如果在這之前沒有對齊：

你之後每一次 Debug，本質上都是在補 Spec。

Chapter 3 的關鍵不是寫 Spec，而是「測 Spec」，你在這一章學的，不是文件技巧，而是一種能力：

判斷：這份 Spec，現在管不管得住 Context？

當你能回答這個問題，你就已經準備好進入 Chapter 4。

如果 Q 開始「問很多問題」，那是好事

很多學生在這一步會很緊張：

- 它怎麼問這麼多？
- 是不是我 Spec 寫得很爛？

其實剛好相反。AI 問問題，代表它不敢亂猜了，這正是 Spec 開始發揮作用的訊號。

3.7 本章你應該達成的狀態

在進入下一章之前，請確認你能做到這幾件事：

- 你知道 Spec 在控制哪一段 Context
- 你知道哪些地方「還不能動」
- 你敢拒絕 AI 幫你加功能
- 你開始用 Spec 來踩煞車

如果你有這些感覺，代表你已經成功升級。

3.8 帶著這份 Spec，進入 Chapter 4

現在你的專案裡已經有三種東西：

1. 可被 AI 讀取的結構
2. 初步但明確的 Spec
3. 被刻意限制的 Context

接下來，我們要做的不是加功能，而是學會怎麼「安全地修改」這個系統。

這正是 Chapter 4 要處理的主題：**Backend First × Iterative × Atomic Edit**，三個讓產品不爆炸的原則。

Chapter 4 產品級三原則

Backend First × Iterative × Atomic Edit 讓系統「開始成長」，但不爆炸

在 Chapter 3，你已經做完一件非常關鍵的事：

你用 Spec，成功鎖住了 AI 的理解範圍。

現在，我們終於可以開始「真的動手」了。

但在你寫下第一行實作之前，你必須先學會一件事：

產品不是寫壞的，是改壞的

這一章要教你的三個原則，就是為了讓你在「開始改很多次」之後，系統還能活著。

4.1 為什麼產品一進實作，就特別容易炸？

在 Week 2，你的系統會開始出現這些特徵：

- 有狀態
- 有多個畫面區塊
- 有重複修改
- 有「之後可能會再加」的誘惑

這代表什麼？Context 會開始高速變動。

如果你沒有一套「改動紀律」，AI 會很努力幫你補齊，但補出來的通常不是你要的產品。

4.2 原則一：Backend First (依賴順序控制)

這不是技術選擇，而是「思考順序」

很多人聽到 Backend First，會以為是在說：

- 先寫 API
- 先接資料庫
- 先搞後端

但在這門課裡，它的真正意思是：

先把「資料與狀態的真相」定下來，再讓畫面去反映它。

在 App #2 中，什麼叫 Backend？

在 Week 2 的範圍內，Backend 指的是：

- data.js 中的資料結構
- 狀態欄位 (status)
- 狀態允許的轉移規則
- Spec 中寫清楚的資料語意

不是伺服器、不是雲端。

為什麼一定要先定 Backend？

如果你先動 UI：

- AI 不知道資料從哪來
- 不知道哪些狀態合法
- 就會開始自己補 API、補邏輯

結果通常是：畫面看起來對，但資料早就亂了。

Backend First 的實作心法

每次你想改畫面前，先問自己一句：

我現在改的，是「資料的真相」，還是「顯示方式」？

- ☐ 如果是真相，先改資料與 Spec。
- ☐ 如果只是顯示，才動畫面。

Chapter 4 | Amazon Q 實作指令 (Backend First)

請先不要修改任何程式碼。

根據 `vibe-coding-week2/spec/app-spec.md`，

請你說明目前系統中：

1. 哪些檔案屬於資料與狀態的「單一真相來源」
2. 哪些檔案只應該負責顯示或操作
3. 如果我要新增狀態相關邏輯，應該先動哪一層

請只做分析，不要產生任何程式碼。

4.3 原則二：Iterative (小步快跑，控制變動幅度)

AI 不是不能做很多事，是不能「一次做很多事」

當你開始實作時，最危險的一句話是：「我們一次把這整段都做好吧。」

為什麼大改會害死產品？

因為對 AI 來說，大改代表：

- 多個 Context 同時變動
- 推理成本暴增
- 無法判斷哪個變動才是你要的

結果通常是：**什麼都做一點，但沒有一個地方是真的對。**

Iterative 的正確定義

小步快跑不是慢，而是每一步都具備三個條件：

- ☐ 改動範圍可描述
- ☐ 結果可驗證
- ☐ 出問題可回滾

在 App #2 中，什麼叫「一步」？

例如：

- 只先處理 todo → in-progress
- 不同時處理所有狀態
- 一次只更新一個 item

Chapter 4 | Amazon Q 實作指令 (Iterative)

根據目前的 Spec，我想先實作「將一個項目從 todo 變為 in-progress」這一個行為。

請你先列出：

1. 這個行為涉及哪些檔案
2. 哪些地方需要被修改
3. 哪些地方現在不應該動

請不要直接產生程式碼。

這一步的目的，是**先框出變動邊界**。

4.4 原則三：Atomic Edit (控制傷害範圍)

產品不是被「寫壞」，而是被「順手改壞」

最常見的災難級指令是：「幫我修一下這裡。」

對你來說很小，對 AI 來說卻是：「那我幫你重構整個世界。」

什麼叫原子性修改？

一句話版本：一次只改一件、只改一處、只改一個原因。

Atomic Edit 的三個必要元素

1. 明確指定檔案
2. 明確指定修改位置
3. 明確限制影響範圍

Chapter 4 | Amazon Q 實作指令 (Atomic Edit 範例)

請對檔案 `src/data.js` 進行修改。

請只將以下內容：`status: "todo"`

替換為：`status: "in-progress"`

請確認：

- 只修改這一處
- 不重構、不調整其他邏輯
- 修改完成後，說明這個變動可能影響的行為

這種指令的目的不是寫程式，而是 保護 Context 不被破壞。

4.5 三原則其實是在做同一件事

現在請你回頭看這三個原則：

- Backend First → 控制「依賴順序」
- Iterative → 控制「變動節奏」
- Atomic Edit → 控制「傷害範圍」

它們其實都在做同一件事：**限制 Context 同時變動的範圍**。

4.6 如果你違反其中一個，會發生什麼事？

違反 Backend First

- UI 看起來正常
- 資料實際已經不一致

違反 Iterative

- 每次改完都不知道是哪裡壞
- Debug 成本暴增

違反 Atomic Edit

- 改一行，壞三個地方
- AI 越幫越忙

4.7 本章結束時，你應該站在這裡

在進入下一章之前，請確認你能做到：

- 你知道現在該先改哪一層
- 你敢拒絕一次做太多事
- 你能精準限制 AI 的修改範圍
- 你開始「管理修改」，而不是只寫功能

如果你有這些感覺，代表你已經真正進入「產品模式」。

4.8 帶著三原則，進入 Chapter 5

現在你的專案已經具備：

- 可被理解的 Spec
- 可被限制的 Context
- 可被安全修改的節奏

接下來，我們要正式把 App #2 從「教學練習」拉升成：一個真的有產品定位的系統。

這正是 Chapter 5 App #2 產品定位 要做的事。

Chapter 5 App #2 的產品定位

從「我做不到」，走向「有人需要」

到目前為止，你已經完成了一件很不容易的事：

- 你沒有被 AI 帶著亂跑
- 你用 Spec 管住了 Context
- 你學會了在改動時保護系統

但現在，我們要面對一個比寫程式更難的問題：

這個 App，到底是拿來給誰用的？

5.1 為什麼「能用」不等於「會被用」？

在技術學習中，很容易出現一個錯覺：只要做得出來，就代表它有價值。但在產品世界裡，價值不是這樣判斷的。

一個殘酷但真實的事實

世界上有大量的系統：

- 功能齊全
- 技術合理
- 沒有人用

原因不是因為它們寫得不好，而是因為：

它們沒有清楚回答「這是為誰存在的」。

5.2 App #2 的真正身份，不是一個看板

表面上，App #2 看起來像是一個：

- 狀態看板
- 任務清單
- 流程管理工具

但如果你只停在這裡，它就會變成：又一個「什麼都能做一點，但沒有靈魂的工具」。

所以這一章，我們要幫 App #2 決定一個明確的產品身份。

5.3 產品定位的第一步：先決定「非誰不可」

在產品定位時，第一個問題不是：我可以加什麼功能？

而是：如果只能有一種人會用，那會是誰？

App #2 在 Week 2 的建議定位

在這門課中，我們給 App #2 一個非常刻意的定位範圍：給「內容型創作者」用的工作流工具。不是所有人，不是企業，不是工程師。

而是這種人：

- 常常有很多內容點子
- 同時在跑多個平台
- 腦中有很多「做到一半的東西」
- 最痛的不是沒想法，而是「卡在中途」

5.4 從功能思維，轉成「場景思維」

當你一旦有了明確使用者，接下來就要換一個腦袋想事情。

功能思維會問的問題

- 要不要加拖拉？
- 要不要加標籤？
- 要不要加分類？

場景思維會問的問題

- 這個人什麼時候會打開 App？
- 他打開時，心裡最急的是哪一件事？
- 他希望一眼看到的是什麼狀態？

你會發現：**場景一出現，很多功能會自動變得「不重要」。**

5.5 App #2 的核心使用場景 (Week 2 版)

在 Week 2，我們只聚焦一個主場景。

核心場景定義

當使用者想知道：「我現在卡住哪些內容？」時，他會打開這個 App。

這代表幾件事：

- 他不是來新增一百筆資料
- 他不是來做精細管理
- 他是來「減少心理負擔」

對系統設計的直接影響

這個場景，會反過來影響：

- 預設顯示順序
- 哪些狀態最顯眼
- 哪些操作是「主操作」

你現在應該能感覺到：產品定位，其實是在幫你「刪功能」。

5.6 這時候，你才有資格問：要不要加功能？

很多人一開始就問：那我可以加 XXX 嗎？

在 Chapter 5，正確的順序是：

1. 使用者是誰
2. 核心場景是什麼
3. 現在最痛的是哪一步
4. 再來才是：功能要不要存在

如果順序顛倒，你會得到一個：功能很多，但沒有方向的系統。

5.7 Chapter 5 的 Amazon Q 使用方式 (產品共識驗證)

在這一章，Amazon Q 的角色再一次轉換。它不再是工程協作，而是用來：驗證「你現在講的產品故事，有沒有自洽」。

Chapter 5 專用 Amazon Q 指令 (產品定位驗證)

請在 `vibe-coding-week2` 專案中，使用以下指令：

請先不要修改任何程式碼。

根據目前的 Spec 與專案結構，請你用「產品使用者」的角度說明：

1. 你認為這個 App 是設計給誰用的
2. 你認為這個使用者什麼情況下會打開它
3. 你認為目前的設計，最想幫他解決的問題是什麼
4. 如果這個 App 今天少了一個功能，哪一個會最影響它的價值？

請只做分析，不要提出新功能建議。

5.8 你現在應該開始有的感覺

當你做到這一步，你可能會開始出現這些想法：

- 原來不是每個功能都重要
- 這個 App 好像真的有一個角色了
- 有些我本來想加的功能，其實現在不需要

這些都是**產品感**開始出現的訊號。

5.9 為什麼我們在 Week 2 停在這裡？

你可能會好奇：為什麼不直接做完一個超完整的產品？

原因很簡單：

因為沒有上線、沒有使用者，產品的價值其實還沒被驗證。

Week 2 的目標不是完美，而是：

- 有清楚定位
- 有穩定結構
- 有可延伸空間

5.10 帶著產品定位，進入 Chapter 6

現在你的 App #2 已經具備：

1. 穩定的 Context
2. 可控的修改節奏
3. 明確的產品對象

接下來，我們要把這些東西收斂成一份「真正可教、可交付的 Spec」。

這正是 Chapter 6 App #2 教學用 Spec 要做的事。

Chapter 6 App #2 教學用 Spec

用 Spec，把「會爆炸的想法」變成「可完成的產品」

到這一章為止，我們要正式完成一件事：

不是再多寫幾行程式，而是把 App #2 定義成「一個做得完的產品」。

這一章的 Spec，不是工程文件，而是你用來 **限制 Context**、**控制實作節奏**、**確保交付** 的工具。

6.1 Spec 使用說明（請先讀）

這份 Spec 採用「學員填空版」設計，請注意三件事：

1. 灰色提示文字請刪掉，換成你的答案
2. 所有標註為 **【A 標準必填】** 的欄位，下課前一定要完成
3. 沒寫清楚的地方，AI 一定會幫你「合理補完」

6.2 Spec 檔案位置（全班統一）

請在專案中建立以下檔案：

```
vibe-coding-week2/  
└─ spec/  
    └─ app-spec.md
```

這份檔案，就是 App #2 的單一真相來源 (Single Source of Truth)。

6.3 App #2 Spec (學員填空版)

請將以下內容完整貼入 app-spec.md，然後逐段填寫。

【一】System Goal (系統目標)【A 標準必填】

請用 2-3 行說明：這個 App「現在這一版」要幫誰解決什麼問題。

1. System Goal

This app is designed for _____.

The main problem it solves is _____.

For Week 2, this app focuses on:

- _____
- _____

【二】User & Core Scenario (使用者與核心場景)【A 標準必填】

想像一個人正在打開這個 App，他為什麼現在需要它？

2. User & Core Scenario

Primary user:

- _____

Core usage scenario:

When the user opens this app, they want to:

- _____

【三】Data Structure (資料結構)【A 標準必填】

不要設計資料庫，只定義「最小可用資料」。

3. Data Structure

Each content item includes:

- id: number
- title: string
- status: string

No additional fields are included in Week 2.

(本段 不可刪改欄位，只能補說明)

【四】State Definition (狀態定義)【A 標準必填】

這一段是整個 App #2 的核心。

4. State Definition

Allowed status values:

- todo
- in-progress
- done

No other status values are allowed.

【五】Allowed State Flow (允許的狀態轉移)【A 標準必填】

請非常明確，因為 AI 會完全照這裡行事。

5. Allowed State Flow

Allowed transitions:

- todo → in-progress
- in-progress → done

Disallowed transitions:

- todo → done
- done → any other state

Each action updates only one item at a time.

【六】UI Scope (畫面範圍)【A 標準必填】

UI 不是畫面美不美，而是「顯示哪些資訊」。

6. UI Scope

The UI includes:

- Three columns: To Do, In Progress, Done
- Each item appears in exactly one column
- Status change triggers immediate list update

UI does NOT include:

- Drag and drop
- Animation effects
- Advanced styling

【七】Out of Scope (本週不做事)【A 標準必填】

這一段是保命用的。

7. Out of Scope

The following features are explicitly out of scope for Week 2:

- User authentication
- Database persistence
- Editing item content
- Custom status values
- Search, filter, or sorting

【八】Success Criteria (完成標準)【A 標準必填】

下課前，你要能「驗收」你的作品。

8. Success Criteria

The app is considered complete for Week 2 if:

- A user can add a new item to To Do
- A user can move an item step-by-step to Done
- Items never appear in more than one column
- Disallowed state transitions are prevented

6.4 這份 Spec 在教你什麼？

請你停下來想一件事：剛剛你沒有寫任何一行程式碼，但這個 App 已經被「鎖定成一個可完成的產品」。

這就是 Spec-driven 的威力。

6.5 Chapter 6 專用 Amazon Q 指令 (Spec 驗收)

在你完成 `app-spec.md` 填寫後，請對 Amazon Q 下這個指令：

請先不要修改任何程式碼。

請你閱讀 `vibe-coding-week2/spec/app-spec.md`，

並檢查這份 Spec 是否已足夠支援一個

「可完成的 Week 2 MVP」。

請說明：

1. 哪些規則已經足夠清楚
2. 是否還有可能產生多種解讀的地方
3. 是否有任何內容超出 Week 2 的實作範圍

請只做驗收與說明，不要補功能、不寫程式碼。

6.6 為什麼我們用「學員填空版」？

因為真正的學習不是抄答案，而是：

- 你知道哪裡該填
- 你知道哪裡不能亂填
- 你知道哪裡一填錯，產品就會炸

6.7 Chapter 6 結束時，你應該站在這裡

在離開 Week 2 之前，請確認你現在可以說出：

- 這個 App 是給誰用的
- 它現在解決哪一個核心問題
- 為什麼有些功能「現在不能做」
- 如果要部署，這個 Spec 已經站得住腳

如果你能做到這些，你已經不是在「學寫 App」，而是在 **打造產品底盤**。

6.8 下一步預告 (Week 3)

下一週，我們會做三件事：

1. 把這個真正 App 上雲 (Cloud Run / Firebase Flow)
2. 理解部署為什麼也是產品的一部分
3. 開始討論：這個 App 要不要、怎麼商業化

Chapter 7A 本地展示與交付

讓 App #2 可 Demo、可驗收、可交接（不部署版）

到這一章為止，你要先接受一個事實：不是所有產品，都應該在「第一次完成」就急著上線。

在 Week 2，我們刻意不做雲端部署，因為這一週的任務只有一個：確認你的產品「站得住腳」。

7A.1 為什麼 Week 2 不急著部署？

部署會帶來什麼？

- 帳號與權限設定
- 指令與工具學習成本
- 非產品本身的 Debug

這些都會：

- 打斷產品思考節奏
- 稀釋「狀態 × 資料一致性」的學習重點

所以 Week 2 的原則是：先確定產品能活，再決定要不要讓它被全世界看到。

7A.2 Week 2 的「完成定義」是什麼？

請記住：Week 2 的完成，不是「上線」，而是「可驗收」。

A 標準（全班必達）

你的 App #2 必須能做到：

- 使用 Live Server 在本地端正常運作
- 可以新增一筆內容到 To Do
- 可以依規則推進狀態（todo → in-progress → done）
- 卡片永遠只存在於一個欄位
- 禁止的狀態轉移會被阻止

如果你做到這些，就代表：你的產品核心已經成立。

7A.3 本地展示方式（Live Server）

在 Week 2，展示方式統一採用：

- VS Code + Live Server
- 或任何等效的本地 HTTP server

展示前檢查清單（請逐項確認）

- 專案資料夾名稱為 `vibe-coding-week2`
- `index.html` 可正常開啟
- 操作流程不需要重新整理頁面
- 狀態改變後，列表即時更新

7A.4 Demo Script (學員展示用)

請你練習用 60–90 秒 展示你的產品，順序如下：

1. 一句話介紹這是什麼產品
 - 給誰用
 - 解決什麼問題
2. 新增一筆內容
 - 出現在 To Do
3. 推進狀態
 - To Do → In Progress
 - In Progress → Done
4. 說明一條「不能做的事」
 - 為什麼 todo 不能直接 done
 - 為什麼 done 不能再改

這個 Demo Script 的目的不是炫技，而是：證明你的 Spec 真的在控制產品行為。

7A.5 交付內容清單 (Week 2 交件包)

Week 2 的交付不是一個網址，而是一個可交接的產品資料夾。

請確認你有以下內容：

```
vibe-coding-week2/  
├─ index.html  
├─ src/  
├─ spec/  
|   └─ app-spec.md  
└─ README.md
```

README.md (建議內容)

你的 README 至少要說清楚三件事：

- 這是什麼 App
- 目前支援哪些行為
- 哪些功能刻意不包含

這會直接影響下一週「要不要部署」。

7A.6 為什麼「交付資料夾」很重要？

因為產品在進入部署前，必須先能：

- 被別人打開
- 被別人理解
- 被別人接手

如果你現在交出去的東西：

- 只有畫面，沒有規則
- 只有功能，沒有 Spec

那就算上線，也很快會失控。

7A.7 Week 2 結束時，你應該站在這裡

在進入 Week 3 前，請確認你可以回答這些問題：

- 這個 App 是給誰用的？
- 它現在解決哪一個核心問題？
- 哪些功能是刻意不做的？
- 如果要上線，還缺哪一塊？

如果你能清楚回答，代表你已經完成 Week 2 的任務。

Chapter 7B (前導) 上線與商業化的準備

給「已經準備好往前衝」的你

有些學員在 Week 2 結束時，會出現這個狀態：

我做完了，而且我知道下一步想幹嘛。

這一節，就是為你準備的。

7B.1 先講結論：部署不是只有一種路

在這門課中，我們會介紹兩條路線：

路線一：Firebase Hosting (主線) Week 3 全班主線

- 適合快速上線
- 幾乎零後端知識
- 有免費層
- 非常適合 App #2 的 MVP

路線二：Cloud Run (進階選配) 示範 + 進階學員加速道

- 適合需要後端邏輯或未來擴展
- 彈性高
- 需要 GCP 專案與帳務設定

7B.2 為什麼我們先選 Firebase Hosting ?

因為產品在早期最需要的是：

- 一個網址
- 可以分享
- 可以收回饋

而不是複雜架構。

Firebase Hosting 剛好符合：

- 靜態前端
- 快速部署
- 心流不中斷

7B.3 Cloud Run 在產品路線中的角色

Cloud Run 不是拿來「取代」 Firebase 的，而是：當你的產品開始需要「邏輯與彈性」時的舞台。

你現在不一定需要它，但你應該知道它存在。

7B.4 從部署走向商業化 (Week 3 將做的事)

下一週，我們會一起完成：

- 把 App #2 上線
- 理解部署如何影響產品決策
- 討論商業模式可能性
 - 內部工具
 - 個人 SaaS
 - 客戶專案
 - 訂閱或一次性收費

7B.5 給進階學員的一句話

如果你現在已經做到：

- App #2 穩定
- Spec 清楚
- 想擴充功能

那你已經站在：可以開始「做產品實驗」的位置了。

Week 2 為什麼不急著上線？

我們這一週沒有急著上線，是因為我們在做更重要的事：把產品底盤打穩。

下一週，我們會把它推出去，看看這個產品，值不值得被留下來。

Chapter 8 狀態升級與互動進階

當產品開始被使用，互動為什麼一定要升級？

在 Chapter 6，你已經完成一個非常重要的里程碑：你有一個穩定、可驗收、可交付的產品 MVP。

它現在可以：

- 正確顯示三個狀態
- 保證資料一致
- 防止不合法的狀態轉移

這代表什麼？你已經解決了「系統會不會爆炸」的問題。

現在，我們要進入下一個層次：系統不再只是能用，而是要開始「好用」。

8.1 為什麼互動升級一定要「延後」？

很多人做 App 時，會有一個衝動：現在能動了，我要讓它更順、更漂亮、更直覺。

這個衝動是對的，但時機如果錯，產品會直接死亡。

如果你在 Chapter 4 之前就做互動升級

會發生三件事：

- 狀態邏輯還沒穩
- UI 變成邏輯來源
- Debug 時你會完全分不清「錯在資料還是錯在畫面」

所以我們刻意把 Chapter 8 放在這裡。

8.2 互動升級的第一個關卡：狀態「回退」要不要存在？

目前你的 App #2 是單向的：todo → in-progress → done。

這非常安全，但你一定會遇到一個真實需求：做到一半，發現要退回去怎麼辦？

產品視角，而不是技術視角

這不是技術問題，而是產品決策：

- 回退會不會造成混亂？
- 使用者會不會誤觸？
- 是否需要確認機制？

在 Chapter 8，我們將：

- 示範「可以怎麼設計回退」
- 但不要求全班實作

因為一旦加入回退，系統複雜度會明顯上升。

8.3 狀態升級的核心原則（仍然適用三原則）

即使進入互動升級，你前面學了三原則 一條都不能丟。

Backend First（再次出現）

回退不是 UI 行為，而是：

- 狀態轉移規則改變
- Allowed Flow 必須更新
- Spec 必須先動

永遠先改 Spec，再改行為。

Iterative (一次只加一種互動)

不要一次做：

- 回退
- 拖拉
- 動畫

請一次只升級一個維度。

Atomic Edit (避免 UI 重構災難)

互動升級時，AI 最愛做一件事：那我順便幫你重構整個 render 邏輯吧。

這是災難來源。

8.4 拖拉 (Drag & Drop) 為什麼是「最後才加」？

拖拉在視覺上很吸引人，但它其實是：狀態轉移的另一種輸入方式。

它不是新功能，而是：

- 把「按鈕推進」
- 換成「手勢推進」

產品設計上的陷阱

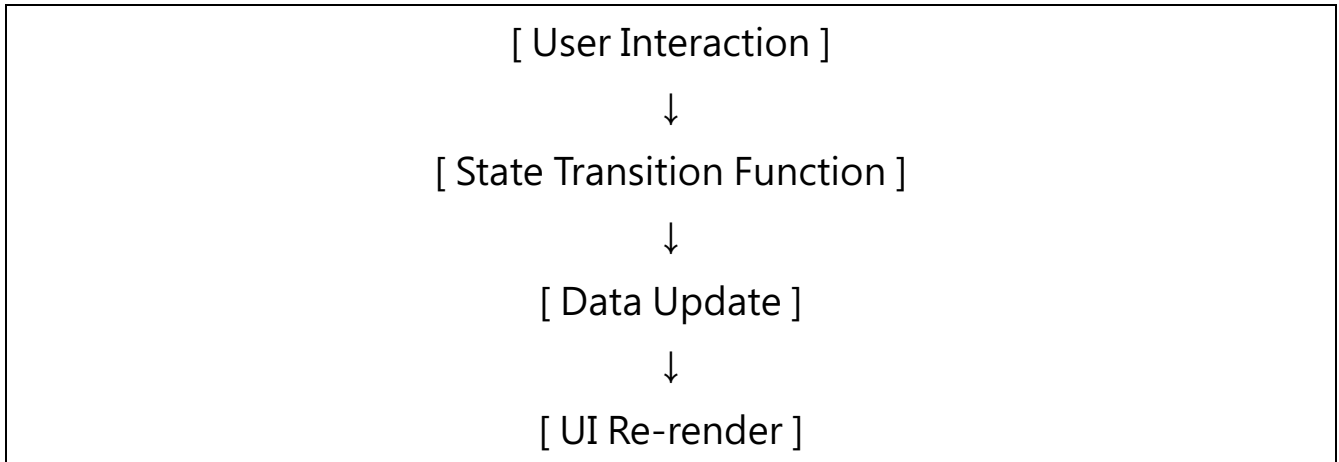
如果你先做拖拉：

- 使用者會繞過你的驗證流程
- 不合法狀態更容易發生
- UI 行為會反過來支配邏輯

正確順序 (非常重要)

1. 先有明確 Allowed State Flow
2. 再讓任何互動「呼叫同一個狀態轉移函式」
3. 拖拉只是其中一個觸發方式

8.5 示範架構：互動 ≠ 狀態邏輯



互動只是入口，狀態轉移才是產品核心。

8.6 Chapter 8 的 Amazon Q 使用方式 (示範用)

這一章不要求全班跟做，Amazon Q 的角色是：設計顧問。

Chapter 8 示範用 Q 指令 (不實作)

請先不要修改任何程式碼。

假設我想在目前的 App #2 中加入「狀態回退」功能，但仍然必須遵守 Spec-driven 與三原則。

請說明：

1. Spec 中哪些地方一定要先修改
2. 狀態轉移規則會增加哪些複雜度
3. 哪些 UI 行為應該共用同一套狀態轉移邏輯
4. 如果實作順序錯誤，最容易出現哪些風險

請只做設計分析，不要提供實作細節。

升級前，用 Amazon Q 做「破壞模擬」。在 Chapter 8，我們不急著升級功能，而是先確認一件事：如果我真的加了這個互動，會破壞哪些東西？

這時候，Amazon Q 的角色不是工程師，而是風險預測員。

Chapter 8 專用 Amazon Q 指令 (風險分析，不實作)

請先不要修改任何程式碼。

目前 App #2 已完成 Week 2 的 MVP，具備穩定的狀態流與資料一致性。

假設我想加入「互動升級」(例如狀態回退或拖拉操作)，請你站在「產品穩定性」的角度分析：

1. 哪些既有規則最容易被破壞？
2. 哪些 Spec 條款一定要先調整？
3. 哪些 UI 行為如果直接實作，會繞過原本的狀態驗證？
4. 哪些地方如果修改順序錯誤，最容易造成系統失控？

請只做風險與設計層級分析，不要提供任何實作或程式碼。

你現在應該學會三件事：

1. 升級不是加功能，是改規則
2. 規則改錯，比功能寫錯更危險
3. 會先問「會壞什麼」的人，才有資格升級系統

這是產品級工程師思維，不是工具使用技巧。

8.7 這一章，你要「帶走什麼」？

不是拖拉程式碼，而是這三個認知：

- 所有互動，都是狀態轉移的不同入口
- UI 永遠不該成為狀態真相
- 功能升級，本質是 Spec 的進化

如果懂這三件事，回家自己加拖拉，成功率會高很多。

8.8 Chapter 8 的定位總結

這一章的角色是：讓你看到產品「往上長」的可能性，而不是逼你們現在就長到極限。這正是產品升級的正確順序。

8.9 銜接下一章

當互動開始升級，下一個一定會遇到的問題是：我關掉瀏覽器，資料怎麼不見了？

這不是 bug，而是產品進入 資料持久化階段 的訊號。

這正是 Chapter 9 資料持久化與多裝置 要處理的事。

Chapter 9 資料持久化與多裝置

為什麼「資料存起來」會讓產品進入下一個世界？

在 Chapter 8，你已經看到一件事：

一旦系統開始升級互動，使用者就會期待「東西不要消失」。

關掉瀏覽器資料不見，不是 bug，而是系統尚未進入產品下一階段的訊號。

9.1 為什麼資料持久化不是「順手加一下」？

很多人會直覺這樣想：那我就把資料存起來就好了啊。

但這一行看似簡單的決定，其實會同時改變三件事：

1. 資料的生命週期
2. 系統的責任邊界
3. 產品對使用者的承諾

這就是為什麼「存資料」一定要是一章，而不是一行 code。

9.2 資料持久化，會讓系統多背一個責任

在目前的 App #2 (Week 2 MVP) 中：

- 資料存在記憶體
- 重整頁面就消失
- 使用者心理預期低

這時系統只需要負責：現在這一次操作是對的。

一旦你加上資料持久化，使用者會自動預期：

- 下次打開還在
- 換裝置也還在
- 不會莫名其妙壞掉

這代表：系統開始要對「時間」負責。

9.3 三種常見的資料持久化層級（產品視角）

在這一章，我們用「產品成熟度」來看資料存放方式。

層級一：localStorage（個人裝置）

- 優點
 - 幾乎零門檻
 - 不需帳號、不需後端
 - 非常適合 MVP 驗證
- 代價
 - 只能在同一台裝置
 - 清除瀏覽器資料就沒了

適合：個人工具、快速驗證

層級二：雲端資料庫（單人帳號）

- 優點
 - 跨裝置
 - 可以備份
 - 為商業化鋪路
- 代價
 - 需要帳號與權限
 - 系統複雜度上升

適合：個人產品、早期 SaaS

層級三：多人協作資料層

- 優點
 - 團隊協作
 - 權限控管
- 代價
 - 架構與成本明顯上升

不在本課程範圍內

9.4 在 App #2，我們為什麼「只示範 localStorage」？

這是教學設計的刻意選擇。如果在這一章直接學：

- Firebase
- SQL
- API

你會以為：資料存起來 = 技術選型問題。

但事實是：資料存起來 = 產品承諾升級問題。

localStorage 的價值

它讓你：

- 專注在「資料何時寫入、何時讀取」
- 不被帳號與權限分心
- 看清楚「資料流」本身

9.5 資料持久化，對 Spec 的影響

這一節非常重要，請你慢慢看。一旦資料會被保存，Spec 中必須多回答三件事：

1. 什麼時候寫入？
2. 什麼時候讀取？
3. 哪些狀態下不應該覆寫？

如果這三件事沒寫清楚，AI 會幫你「好心補完」，然後系統就會亂。

9.6 Amazon Q 使用方式 (技術選型顧問)

在這一章，Amazon Q 的角色是：幫你理解「不同持久化選擇，會帶來什麼產品後果」，不是幫你寫程式。

Chapter 9 專用 Amazon Q 指令 (選型分析)

請先不要修改任何程式碼。

目前 App #2 是一個單人使用的內容管理工具，具有明確的狀態流與資料一致性規則。

請你從「產品責任」的角度分析：

1. 使用 localStorage 作為資料持久化，系統需要承擔哪些責任？
2. 如果未來改成雲端資料庫，哪些 Spec 條款一定要調整？
3. 哪些情況下不應該在 Week 2 就導入雲端資料層？

請只做分析與比較，不要提供實作方式。

9.7 示範段：localStorage 接入的正確順序 (講師示範)

這一段以示範為主，不要求全班跟做。

示範順序 (只講概念，不貼完整程式碼)：

1. 資料初始化 → 先讀 localStorage
2. 狀態改變 → 更新記憶體資料
3. 同步寫回 localStorage
4. UI 根據資料重新渲染

localStorage 不是資料來源，它只是資料的「備份層」

9.8 常見錯誤示範

- 每次 render 都覆寫 localStorage
- UI 狀態跟資料狀態不一致
- 把 localStorage 當成即時資料來源

這些錯誤，幾乎都是：沒有先想清楚「資料生命週期」。

9.9 本章結束時，你應該理解的三件事

在進入下一章之前，請確認你理解：

1. 為什麼資料持久化是產品層級的決定
2. 為什麼 localStorage 是安全的第一步
3. 為什麼 Spec 必須跟著資料層一起升級

如果你理解這三件事，你已經準備好面對「真的使用者」。

9.10 銜接下一章

當資料開始被保存，下一個一定會出現的問題是：

- ☐ 那使用者怎麼用？
- ☐ 我怎麼知道他在幹嘛？

這不是技術問題，而是**產品回饋與行為分析**的問題。

這正是 **Chapter 10 上線後的產品調整** 要處理的事。

Chapter 10 真實上線後的產品調整

為什麼「能跑」不是終點，而是你第一次被現實打臉的開始

在 Chapter 9，你已經讓產品具備了一個重要能力：資料可以留下來。

這代表一件事：你的產品，已經開始對「未來」負責了。

而一旦資料留下來，下一個一定會發生的事情是：使用者會開始用出你沒想過的方式。

10.1 上線後第一個會壞的，通常不是技術

很多人以為上線後會遇到的問題是：

- 程式錯誤
- 效能不夠
- 架構不穩

但實務上，第一個出現的通常是：

- 使用者看不懂
- 使用方式跟你想的不一樣
- 他卡住，但你不知道卡在哪

這不是 bug，是產品理解落差。

10.2 「回饋」不是意見，是行為的痕跡

請你記住這句話：真正有價值的回饋，不是使用者說了什麼，而是他做了什麼。

在 App #2 這種產品裡，行為痕跡可能是：

- 卡片長時間停在某個狀態
- 很少有人把項目推到 Done
- 使用者只新增，不整理

這些都不是技術錯誤，而是產品訊號。

10.3 為什麼這些問題「不能直接改程式」？

當你看到這些現象時，很容易下意識想：那我就幫他加個功能吧。

這是產品死亡最快的路。

因為你現在面對的不是功能缺失，而是：

- 流程是否清楚
- 狀態是否符合使用者心理模型
- Spec 是否反映真實使用情境

10.4 上線後的正確調整順序（非常重要）

在這一章，你要養成一個產品級反射動作。正確順序是：

1. 觀察行為
2. 整理假設
3. 修改 Spec
4. 再調整實作

不是反過來。

為什麼一定要先動 Spec？

因為一旦你跳過 Spec：

- AI 會以舊世界觀幫你改
- 你會得到「看似合理、實際混亂」的結果
- 每改一次，Context 都會變得更不穩

10.5 Spec 在這一章的角色改變了

在前面幾章，Spec 的角色是：護欄，到了 Chapter 10，它多了一個新角色：產品紀錄與決策日誌。

Spec 不只是「現在規則是什麼」，而是：

- 為什麼當初這樣設計
- 什麼情況下我們決定要改
- 改動的目標是什麼

10.6 Amazon Q 使用方式 (回饋分析員)

在這一章，Amazon Q 不寫程式。它幫你做的是：把混亂的使用現象，整理成可討論的產品問題。

Chapter 10 專用 Amazon Q 指令 (回饋 → 假設)

請先不要修改任何程式碼。

假設 App #2 已經被實際使用一段時間，目前觀察到以下現象：

- 很多項目停留在 in-progress
- 很少項目進入 done
- 使用者很少回頭整理舊項目

請你從「產品設計」的角度分析：

1. 這些行為可能代表哪些使用者困惑或心理阻力？
2. 哪些 Spec 條款可能需要被重新檢視？
3. 有哪些調整是「流程或規則層級」，而不是功能層級？

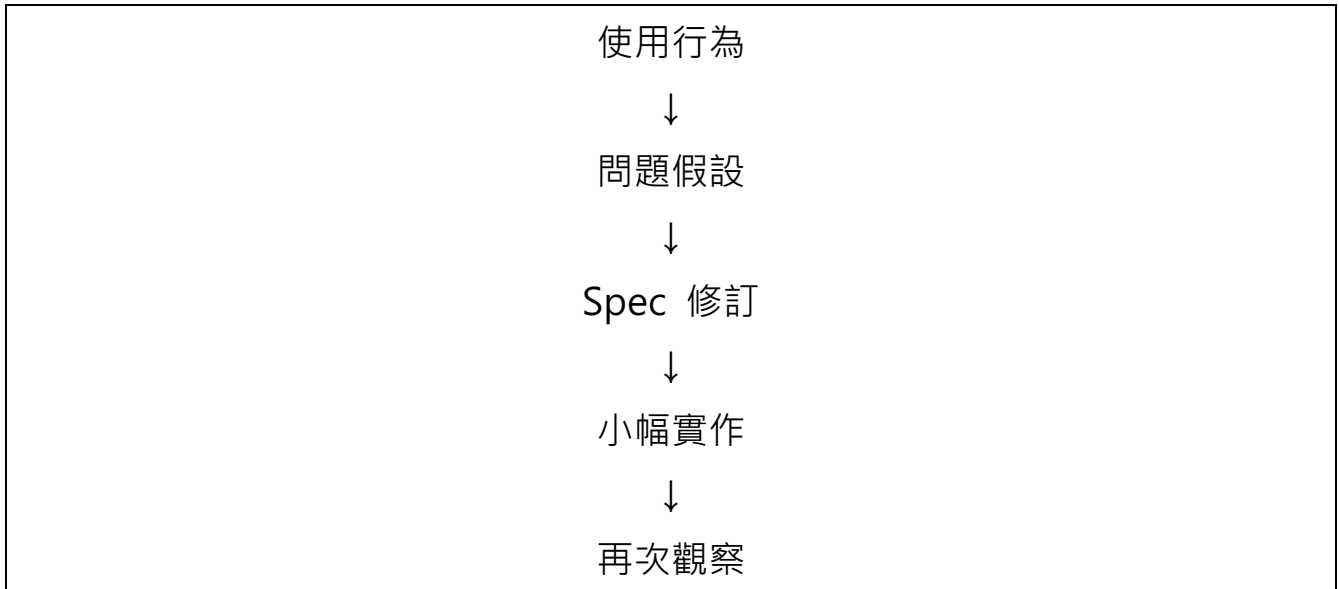
請只做分析，不要提出具體功能實作。

10.7 常見的「錯誤修法」

- 一看到卡住，就加提醒
- 一看到不用，就刪功能
- 一有抱怨，就改 UI

這些做法的共同問題是：它們在逃避真正的產品問題。

10.8 一個健康的產品調整循環



這個循環，才是：AI 能長期穩定協作的前提。

10.9 為什麼這一章，是 AI 課程的分水嶺？

大多數 AI 寫程式課，只教到：怎麼把東西做出來，但真正的產品工作，發生在：做出來之後怎麼不亂改。

這一章教的，是：

- 克制
- 判斷
- 延遲滿足

這些能力，才是你之後能「用 AI 做大事」的關鍵。

10.10 銜接最後一章

現在，你已經具備了：

- 能做出產品的能力
- 能穩定修改的紀律
- 能從回饋中演化 Spec 的方法

接下來最後一個問題是：那 AI，未來到底該扮演什麼角色？

- ☐ 是工具？
- ☐ 是工程師？
- ☐ 還是系統的一部分？

這正是 **Chapter 11 AI × 產品的長期協作模式** 要回答的問題。

Chapter 11 AI × 產品的長期協作模式

從「我叫 AI 幫我寫」，到「我們一起維持一個系統」

走到這一章，你其實已經不知不覺完成了一次身份轉換。一開始你可能是這樣想的：我想用 AI 快一點把東西做出來。但如果你真的一路跟到這裡，你現在腦中多半變成這樣：我要怎麼讓這個系統，在未來半年、一年，都還能被穩定修改？這一章，就是在回答這個問題。

11.1 AI 在產品生命週期中的三種角色

AI 不只是一個角色，它會隨著產品階段改變定位。

第一階段：加速器 (Builder)

發生在你剛開始時：

- 幫你快速生成骨架
- 降低啟動門檻
- 把模糊想法變成可運作的東西

但如果 AI 永遠只停在這一層，產品一定會炸。

第二階段：協作者 (Collaborator) 這一層，產品才開始穩定。

這是你在 Chapter 3–7 學到的狀態：

- AI 需要被 Spec 限制
- 修改有順序、有範圍
- AI 不再「自由發揮」，而是「在護欄內工作」

第三階段：系統的一部分 (System Component)

- AI 不只是來寫 code
- 它參與的是「理解、判斷、演化」
- 它被固定在流程中，而不是臨時被叫來救火

11.2 真正成熟的協作，不是 Prompt，而是「位置」

請你記住這句話：不是 Prompt 寫得好，而是 AI 被放在對的位置。

回顧你這門課中 AI 出現的位置：

- Chapter 2：看 Context
- Chapter 3：對齊 Spec
- Chapter 4：受控修改
- Chapter 8：風險預測
- Chapter 10：回饋分析

你會發現一件事：AI 從來沒有被放在「決策者」的位置。這是關鍵。

11.3 把 AI 從「救火隊」拉回「標準流程」

很多人使用 AI 的方式是：

- 卡住了 → 叫 AI
- 爆炸了 → 叫 AI
- 看不懂 → 叫 AI

這會讓 AI 永遠處在：高 Context、低理解、亂補完的狀態。

AI 的正確進場時機

- 在改之前 → 幫你預測風險
- 在寫 Spec 時 → 幫你對齊理解
- 在上線後 → 幫你整理回饋

而不是：

- 在系統已經亂掉之後，
期待它幫你「全部修好」。

11.4 Spec × AI：一個會一起老的組合

如果你只帶走一個長期做法，請帶走這個：

Spec 不是寫給人看的，而是寫給「未來的你 × AI」看的

為什麼？因為半年後：

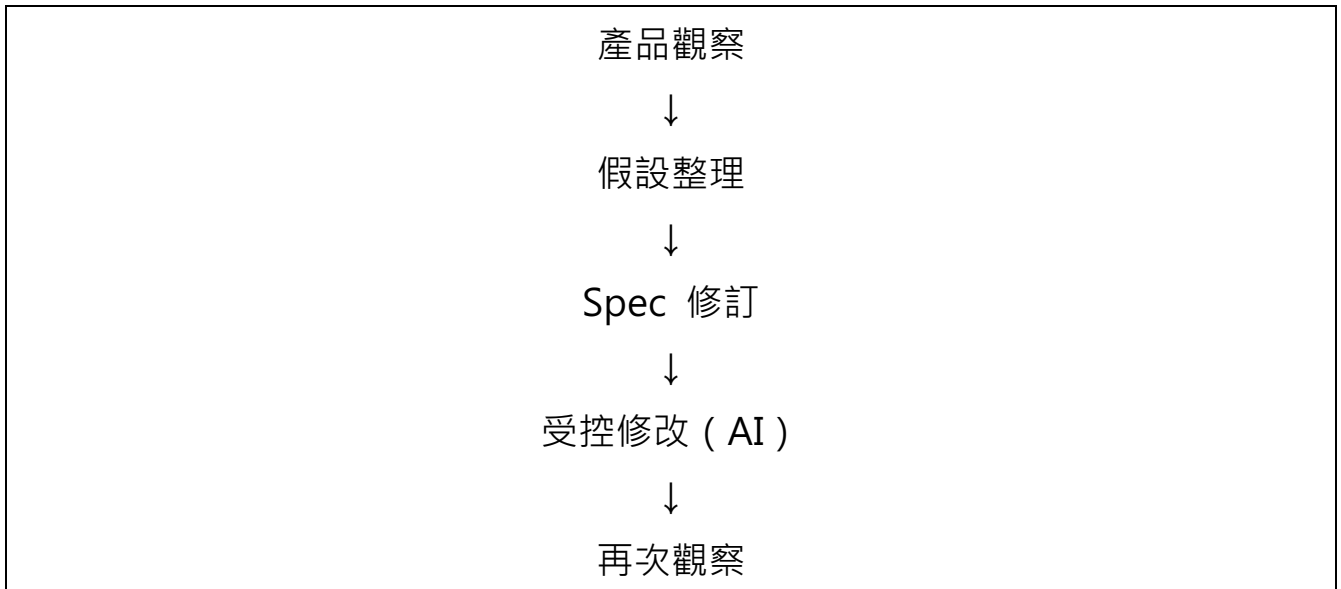
- 你會忘記當初為什麼這樣設計
- AI 也只看得到現在的檔案

Spec 是唯一能：

- 保留設計理由
- 限制 Context 演化方向
- 防止產品變形的東西

11.5 長期協作的最小閉環（請記住這個循環）

這個循環，是你未來所有 AI 協作的底盤。



如果哪一天你發現：

- 每次都直接跳到「改程式」
- Spec 永遠追不上實作
- AI 改完你不敢動

那代表你已經脫離這個循環了。

11.6 為什麼這一章，才是真正的「AI 素養」

很多人把 AI 素養理解成：

- 會不會下指令
- 會不會用新工具
- 會不會寫 Prompt

但你現在應該知道了：**真正的 AI 素養，是系統思維。**

是你能不能：

- 判斷什麼時候該讓 AI 出手
- 判斷什麼時候該先停下來想
- 判斷哪些東西不能交給 AI

11.7 一段現實提醒（很重要）

這門課教的不只是：用 AI 做一個 App，而是：用 AI 長期維持一個會成長的**產品**。

這代表你未來一定會遇到：

- 想快一點的誘惑
- 想一次改很多的衝動
- 想把麻煩全部丟給 AI 的時候

這些時候，請回到這 11 章。

11.8 你現在站在哪裡？

如果你走到這裡，你已經具備：

- 把想法變成產品的能力
- 用 Spec 管住 Context 的能力
- 讓 AI 成為長期協作者，而不是隨機工具的能力

這不是初學者的位置。

11.10 真正的結尾（不是下課）

最後，我想留你一個開環問題，當作這門課的結束：

如果未來 AI 越來越強，那什麼東西，反而一定要留在人類手上？

如果你心中已經有答案，那這門課的目的，就已經達成了。

Week 2 總結：我們這一週到底做了什麼？

如果你現在腦袋有點滿，這是正常的。因為第二週，我們做的事情不是「多寫幾行程式」，而是跨過一個關鍵門檻。

這一週，你已經從：「讓 AI 幫我把東西寫出來」走到「我開始知道，什麼時候該讓 AI 動手，什麼時候不該。」

一、Week 2 的真正主題，不是功能

這一週看起來我們做了很多事：

- App #2
- 狀態管理
- Context / Spec
- localStorage
- Demo / 交付

但其實，所有內容都圍繞同一件事：

當系統開始有狀態、有資料、有未來，你要怎麼避免它失控？

二、你在 Week 2 學到的三個關鍵能力

① 你知道 AI 為什麼會「越改越亂」

你已經理解：

- AI 是 Context-driven 的
- 專案一長，Context 就會變複雜
- Token 爆炸，其實是「世界觀不穩定」的症狀

這代表一件事：你不再怪工具，你會回頭檢查結構。

② 你知道 Spec 的真正用途

在 Week 2，你應該已經不會再把 Spec 當成：

- 文件
- 作業
- 給老師看的東西

而是開始把它當成：限制 AI 思考方式的護欄

你也實際用過 Spec 來：

- 對齊理解
- 控制修改範圍
- 防止過度補完

③ 你完成了一個「可交付」的產品

請你不要小看這一點。在 Week 2 結束時，你至少已經做到：

- 一個可以 Demo 的 App
- 一個有狀態規則的系統
- 一個能被別人打開理解的專案資料夾

這代表你已經不是在「玩範例」。

三、為什麼我們刻意停在「本地 Demo」？

你可能會想：「都做到這樣了，為什麼不直接上線？」

原因只有一個：我們在確認這個產品，值不值得被推出去。

在沒有這一週的訓練之前：

- 上線只會放大混亂
- 部署只會讓 Debug 更痛苦

現在，你至少已經知道：

- 哪些規則是穩定的
- 哪些地方還不能亂動
- 哪些功能其實可以先不要

四、如果你現在有這些感覺，是對的

如果你現在心裡有出現以下念頭，恭喜你：

- ☐ 我好像開始會忍住，不急著加功能
- ☐ 我會先想 Spec，再想怎麼改
- ☐ 我會懷疑，這是不是該現在就做的事

這代表你正在從使用者，轉變成管理者。

Week 3 預告

下一週，我們要把「作品」推向「真實世界」

第二週，我們解決的是：系統會不會失控？

第三週，我們要面對的是：這個產品，有沒有存在的價值？

Week 3 你會做的三件事

① 把 App #2 推出教室

你會學到：

- 為什麼「上線」本身就是產品決策
- Firebase Hosting / Cloud Run 各自代表什麼選擇
- 免費層可以做到哪裡，限制在哪

重點不是工具，而是：你選的部署方式，會影響產品未來長什麼樣子。

② 開始思考「這是不是一個產品」

第三週，我們會討論：

- 這個 App 是誰會用？
- 是一次性工具，還是可以長期存在？
- 是個人使用，還是可以賣給別人？

你會第一次用「產品視角」回看自己做的東西。

③ 學會「不亂改」的長期協作方式

第三週的重點不是再加功能，而是：

- 上線後該怎麼觀察
- 收到回饋該怎麼改 Spec
- AI 要放在哪個位置，才不會變成災難

給你的一句提醒 (Week 2 → Week 3)

第二週，你學的是「穩定」。

第三週，你學的是「選擇」。

穩定，讓你撐得住；選擇，才會決定你走多遠。

最後一句

如果你現在覺得：「原來寫程式不是最難的，最難的是不要亂動。」

那你這一週，真的學到重點了。

附錄：一頁 Week 2 → Week 3 能力轉換圖

你不是在多學技術，而是在換位置

請你先看清楚這件事：Week 2 到 Week 3，不是「再學更多」，而是「站到更高的位置」。

能力轉換總覽（請你慢慢對照）

面向	Week 2：系統穩定期	Week 3：產品選擇期
核心關注	系統會不會失控	這東西值不值得存在
你的角色	系統管理者	產品決策者
關鍵能力	用 Spec 管住 Context	用選擇定義產品方向
AI 的角色	受控協作者	分析顧問 + 長期夥伴
成功標準	能 Demo、能交付	能上線、能說清楚價值
主要風險	改太多、亂補完	做錯方向、浪費時間

思考層級的轉換

Week 2 你在想的是：

- 我這樣改會不會壞？
- AI 有沒有亂補？
- Spec 有沒有守住？

Week 3 你要開始想的是：

- 這個功能真的需要嗎？
- 上線後誰會用？
- 免費層夠不夠？
- 如果有人用，我準備好承擔後果了嗎？

你從「把事情做對」，進化成「決定要不要做」。

為什麼一定要經過 Week 2，才能進 Week 3？

因為沒有 Week 2 的人：

- 上線只會放大混亂
- 部署只會讓 Debug 更痛苦
- 商業化只會變成空想

而你現在至少已經做到一件事：你有一個不會自己爆炸的系統。

這是進入 Week 3 的入場券。

Week 2 教你「撐得住」，Week 3 才開始決定「走不走得遠」。

附錄：Week 2 課後反思題

請你先想清楚，再繼續往前

這不是作業，也不會收。但如果你真的想把這個 App 帶出教室，請你誠實回答自己。

反思題一：關於你的 App #2

請你試著回答，不超過三句話：

1. 這個 App 是給誰用的？
2. 它現在解決的是哪一個明確問題？
3. 如果只能留一個功能，會是哪一個？

如果你回答不出來，代表你還沒準備好進 Week 3。

反思題二：關於你「忍住沒做的事」

請你回想這一週，你刻意沒有做的是什麼？

- 沒加的功能
- 沒優化的 UI
- 沒立刻上的部署

請寫下一個，並回答：為什麼現在不做，反而是對的？

這一題，沒有標準答案，但很重要。

反思題三：關於 AI

請你誠實想一想：

- 這一週，有沒有哪一次你很想叫 AI「直接幫我改完」？
- 那一次，你最後是忍住了，還是放手了？
- 結果有什麼差別？

你會在這一題，看見自己成長最多的地方。

附錄：Week 2 課後 Amazon Q 指令卡

這是用來「整理自己」，不是改系統

⚠️ **重要提醒：**以下指令請在不修改任何程式碼的前提下使用。

指令 1：產品定位盤點（必做）

請先不要修改任何程式碼。

請你根據目前 App #2 的狀態，協助我整理以下內容：

1. 這個 App 目前最核心的使用情境是什麼？
2. 它解決的是「痛點」還是「方便」？
3. 哪些功能如果拿掉，產品仍然成立？

請以產品分析的角度回答，不要提供實作建議。

指令 2：上線準備度自我檢查（選做）

請先不要修改任何程式碼。

假設我考慮在 Week 3 將 App #2 上線，請你從「產品風險」的角度協助我分析：

1. 目前最不適合立刻上線的原因是什麼？
2. 哪些問題應該在上線前解決？
3. 哪些問題其實可以上線後再觀察？

請只做風險與判斷分析，不要提出技術解法。

指令 3：Spec 成熟度檢視（進階）

請先不要修改任何程式碼。

請你檢視目前 App #2 的 Spec，
並協助我判斷：

1. 哪些規則已經穩定到可以長期維持？
2. 哪些規則其實還只是暫時假設？
3. 如果進入 Week 3，上線後哪些 Spec 最可能被迫修改？

請以「產品演化」的角度分析。

給學員的最後一句話

第二週，你學會的是克制；第三週，我們才開始談野心。

能走到這裡的你，已經不是在學工具了。